



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



**TFG del Grado en Ingeniería
Informática**

**Clasificación de TDAH a
partir de señales EEG
mediante técnicas de
Machine Learning y Deep
Learning**



Presentado por Adrián Jiménez García
en Universidad de Burgos — 16 de septiembre
de 2026

Tutor: Bruno Baruque Zanon
Cotutor: Jesús Enrique Sierra García



UNIVERSIDAD DE BURGOS
ESCUELA POLITÉCNICA SUPERIOR
Grado en Ingeniería Informática



D. Bruno Baruque Zanon, profesor del Departamento de Digitalización, área de Ciencia de la Computación e Inteligencia Artificial, y D. Jesús Enrique Sierra García, profesor del Departamento de Digitalización, área de Ingeniería de Sistemas y Automática.

Exponen:

Que el alumno D. Adrián Jiménez García, con DNI 21745253H, ha realizado el Trabajo final de Grado en Ingeniería Informática titulado Clasificación de TDAH a partir de señales EEG mediante técnicas de Machine Learning y Deep Learning.

Y que dicho trabajo ha sido realizado por el alumno bajo la dirección de los que suscriben, en virtud de lo cual se autoriza su presentación y defensa.

En Burgos, 16 de septiembre de 2026

Vº. Bº. del Tutor:

Vº. Bº. del co-tutor:

D. Bruno Baruque Zanon

D. Jesús Enrique Sierra García

Resumen

Este trabajo presenta EEGScope, una aplicación web para experimentar con la clasificación de señales EEG de sujetos con TDAH y controles. El proyecto combina la comparación de modelos de aprendizaje automático y profundo con una herramienta que permite cargar datasets, entrenar, consultar experimentos y realizar inferencia con modelos guardados.

La evaluación utiliza particiones por sujeto para evitar que las ventanas de un mismo paciente aparezcan en entrenamiento y prueba. En los resultados guardados, Random Forest obtiene un F1 ponderado medio del 78,72 % y CNN 1D del 80,14 %. Estas métricas se calculan por ventana y las dos ramas utilizan distintas configuraciones de procesamiento, por lo que no constituyen una comparación aislada de las arquitecturas.

La aplicación utiliza FastAPI, React con TypeScript y PostgreSQL. El análisis de datasets y el entrenamiento se ejecutan en segundo plano mediante Celery y Redis, y los recursos se relacionan con usuarios autenticados. Se dispone también de una landing estática y una configuración de publicación mediante Cloudflare Tunnel. El resultado es un prototipo académico para explorar el proceso de clasificación, no una herramienta validada para diagnóstico clínico.

Descriptores

EEG, TDAH, clasificación, aprendizaje automático, aprendizaje profundo, validación por sujeto, aplicación web.

Abstract

This project presents EEGScope, a web application for experimenting with the classification of EEG recordings from subjects with ADHD and controls. It combines a comparison of machine learning and deep learning models with a tool for uploading datasets, training models, reviewing experiments and running inference with saved models.

Evaluation uses subject-wise splits to prevent windows from the same subject from appearing in both training and test sets. In the saved results, Random Forest achieves a mean weighted F1 of 78.72%, while the 1D CNN achieves 80.14%. These metrics are computed per window, and the two approaches use different processing configurations, so the results do not isolate the effect of model architecture.

The application uses FastAPI, React with TypeScript and PostgreSQL. Dataset analysis and training run in the background through Celery and Redis, and resources are linked to authenticated users. The project also includes a static landing page and a deployment configuration using Cloudflare Tunnel. The result is an academic prototype for exploring the classification workflow, not a clinically validated diagnostic tool.

Keywords

EEG, ADHD, classification, machine learning, deep learning, subject-wise validation, web application.

Índice general

Índice general	iii
Índice de figuras	v
Índice de tablas	vi
1. Introducción	1
1.1. Contexto del proyecto	1
1.2. Problema que se aborda	2
1.3. Alcance y resultado del trabajo	2
1.4. Materiales entregados	3
2. Objetivos del proyecto	5
2.1. Objetivo general	5
2.2. Objetivos de la parte experimental	5
2.3. Objetivos funcionales	6
2.4. Objetivos técnicos	6
3. Conceptos teóricos	9
3.1. Señales EEG y organización de los datos	9
3.2. Segmentación en ventanas	11
3.3. Características temporales y espectrales	11
3.4. Modelos de Machine Learning	13
3.5. Modelos de Deep Learning	14
3.6. Validación por paciente	14
3.7. Métricas de evaluación	15
3.8. Importancia de características	15

4. Técnicas y herramientas	17
4.1. Técnicas de desarrollo y evaluación	17
4.2. Procesamiento de datos y entrenamiento	18
4.3. Backend y persistencia	18
4.4. Interfaz web	19
4.5. Entorno y despliegue	19
4.6. Control de versiones y calidad	20
4.7. Uso de inteligencia artificial generativa	20
5. Aspectos relevantes del desarrollo del proyecto	21
5.1. Preparación de los datos	21
5.2. Diseño de los experimentos	22
5.3. Resultados de clasificación	24
5.4. Del experimento a la aplicación	29
5.5. Publicación y comprobación del sistema	33
6. Trabajos relacionados	35
6.1. Características espectrales y clasificación	35
6.2. Potenciales relacionados con eventos	36
6.3. Redes neuronales e interfaces	36
6.4. Arquitecturas para señales EEG	36
6.5. Herramientas de análisis EEG	37
6.6. Relación con el proyecto	38
7. Conclusiones y líneas de trabajo futuras	41
7.1. Conclusiones	41
7.2. Limitaciones	42
7.3. Líneas de trabajo futuras	43
Bibliografía	45

Índice de figuras

3.1.	Primeras 640 muestras de los canales Fz, Cz y Pz del sujeto v10p , equivalentes a cinco segundos a 128 Hz. Amplitud en unidades arbitrarias, conservando los valores del CSV sin conversión a una unidad física ni filtrado o normalización adicional. Elaboración propia a partir del dataset utilizado.	10
3.2.	Ejemplo de segmentación de un registro de un sujeto en tres ventanas solapadas. L indica la longitud de cada ventana y S el desplazamiento entre sus inicios. Esquema de elaboración propia.	12
5.1.	Curvas ROC de los tres clasificadores ML a partir de las predicciones de prueba de la validación cruzada por sujeto.	27
5.2.	Importancia por permutación agregada por canal para Random Forest.	28
5.3.	Importancia por permutación agregada por canal para XGBoost.	28

Índice de tablas

3.1. Las doce características temporales calculadas por canal y ventana. Elaboración propia a partir de la implementación.	12
3.2. Distribución de las características espectrales por ventana. Elaboración propia a partir de la implementación.	13
5.1. Configuraciones utilizadas en los experimentos ML y DL.	24
5.2. Resultados ML por ventana en cinco particiones por sujeto. Media y desviación típica expresadas en porcentaje.	25
5.3. Resultados DL por ventana en cinco particiones por sujeto. Media y desviación típica expresadas en porcentaje.	25
6.1. Comparación del enfoque de distintas herramientas EEG. Elaboración propia a partir de su documentación oficial y de las funciones del proyecto.	37
6.2. Comparación de los enfoques de los trabajos relacionados y EEGScope.	39

1. Introducción

Este Trabajo de Fin de Grado consiste en el desarrollo de EEGScope, una aplicación web para la clasificación de señales EEG entre sujetos con TDAH y sujetos pertenecientes al grupo de control mediante modelos de Machine Learning y Deep Learning. El trabajo tiene dos partes relacionadas: una parte experimental, en la que se preparan los datos, se comparan y se ajustan diferentes modelos, y una parte de desarrollo software, que permite utilizar ese proceso desde una interfaz.

La idea no es quedarse únicamente con un modelo entrenado y una tabla de resultados. La aplicación permite cargar un dataset, consultar sus características, configurar un entrenamiento, guardar el experimento y utilizar el modelo obtenido para realizar nuevas predicciones. De esta manera, se conecta el trabajo de investigación con una herramienta que se puede ejecutar y probar sin tener que modificar directamente los scripts.

1.1. Contexto del proyecto

El trastorno por déficit de atención e hiperactividad, o TDAH, es un trastorno del neurodesarrollo asociado a dificultades de atención, hiperactividad e impulsividad. Estas dificultades pueden afectar a distintos ámbitos de la vida de la persona [12].

En este contexto, existen trabajos que estudian la clasificación de señales de electroencefalografía mediante técnicas de aprendizaje automático. Algunos extraen características de la señal y utilizan clasificadores tradicionales; otros trabajan con redes neuronales y representaciones como los espectrogramas [17, 27]. Este proyecto se sitúa en esa línea de investigación, pero

mantiene una finalidad académica: clasificar un registro de un dataset no equivale a realizar un diagnóstico médico.

1.2. Problema que se aborda

Trabajar con señales EEG no consiste simplemente en pasar un CSV a un clasificador. Es necesario comprobar su estructura, separar los registros de cada sujeto, decidir cómo dividir la señal en ventanas y mantener el mismo procesamiento al entrenar y al realizar inferencia.

Una dificultad importante es la evaluación. Si se reparten las ventanas aleatoriamente, pueden aparecer datos de un mismo paciente tanto en entrenamiento como en prueba. En ese caso, el resultado no respondería bien a la pregunta de cómo funciona el modelo con un paciente nuevo. Por este motivo, el proyecto aplica una separación por sujeto.

También hay dificultades relacionadas con el uso de la aplicación. Un entrenamiento puede tardar bastante más que una petición habitual a una API. Además, al introducir usuarios, los datasets y experimentos no deben mostrarse de forma conjunta a todo el mundo. La solución incluye tareas en segundo plano y comprobaciones de acceso a los recursos.

1.3. Alcance y resultado del trabajo

El resultado es una aplicación con backend en FastAPI, frontend en React y TypeScript, base de datos PostgreSQL y un worker de Celery conectado a Redis. Junto con la aplicación se mantienen los scripts de investigación, los modelos de referencia exportados y los resultados de validación cruzada utilizados antes del desarrollo de esta.

El usuario puede registrarse, iniciar sesión, trabajar con sus datasets, lanzar entrenamientos ML o DL y consultar los experimentos guardados. Para inferencia puede elegir un modelo propio o uno de los modelos de referencia incluidos en el proyecto: el modelo seleccionado de Machine Learning y el de Deep Learning. El despliegue se prepara mediante Docker Compose, Caddy y Cloudflare Tunnel, mientras que la página pública se publica por separado.

El alcance no incluye adquisición de EEG en tiempo real, validación clínica ni un servicio con disponibilidad garantizada. Las predicciones deben interpretarse dentro de las condiciones del dataset y de la evaluación realizada.

1.4. Materiales entregados

El proyecto incluye el código fuente de la aplicación, los scripts de investigación, los resultados experimentales y la documentación. El repositorio contiene las instrucciones para ejecutar el sistema y consultar el trabajo realizado:

- Repositorio del proyecto: <https://github.com/ajg1005/eeg-adhd-clasificacion>.
- Página de presentación: <https://eegscope.dev>.
- Aplicación web: <https://app.eegscope.dev>.

La documentación se entrega en dos documentos: la memoria y los anexos. Estos últimos incluyen la planificación, los requisitos, el diseño y las instrucciones de instalación y uso.

La aplicación publicada depende de que el equipo que aloja los servicios esté encendido y conectado. El repositorio permite ejecutar el sistema en local siguiendo las instrucciones de instalación.

2. Objetivos del proyecto

En este capítulo se recogen los objetivos del trabajo. Se distingue entre el objetivo general, la parte experimental y los objetivos técnicos de la aplicación.

2.1. Objetivo general

Desarrollar una aplicación web de uso académico que permita una clasificación binaria de señales EEG entre TDAH y Control, comparar modelos de Machine Learning y Deep Learning y utilizar los modelos entrenados.

2.2. Objetivos de la parte experimental

- Preparar un pipeline de carga, limpieza y segmentación de registros EEG, conservando el identificador del sujeto al que pertenece cada ventana.
- Extraer características temporales y espectrales para los modelos clásicos y preparar ventanas de señal para los modelos de Deep Learning.
- Comparar varios clasificadores mediante validación cruzada con separación por paciente, evitando mezclar sujetos entre entrenamiento y prueba.
- Evaluar los resultados mediante exactitud, exactitud balanceada, precisión, recall, F1 y curvas ROC, indicando a qué nivel se calcula cada medida.

- Estudiar la importancia de las características del modelo ML seleccionado y de XGBoost, sin interpretar esa importancia como una explicación clínica.
- Exportar los modelos seleccionados junto con la configuración necesaria para reproducir su entrada durante la inferencia.

2.3. Objetivos funcionales

- Permitir la carga y reutilización de datasets y mostrar estadísticas que ayuden a comprobar su estructura antes de empezar el entrenamiento.
- Configurar y ejecutar entrenamientos desde la interfaz.
- Guardar los parámetros y resultados de cada experimento y relacionarlos con el modelo final exportado.
- Permitir la selección de modelos propios y de referencia para realizar predicciones sobre un archivo CSV que contiene las señales EEG de un paciente.
- Ejecutar el análisis de datasets y los entrenamientos en segundo plano para permitir la navegación mientras se completa el cálculo.
- Incorporar registro e inicio de sesión para que cada usuario pueda tener tanto sus datasets como sus modelos entrenados almacenados.

2.4. Objetivos técnicos

- Mantener separadas las responsabilidades del frontend, la API, el procesamiento EEG y la persistencia, de forma que se pueda modificar una funcionalidad sin tener que reorganizar el resto del proyecto.
- Facilitar la ejecución en otro equipo mediante Docker Compose y ficheros de dependencias versionados.
- Añadir pruebas automatizadas sobre los comportamientos principales y utilizar integración continua para detectar errores antes de integrar los cambios.
- Documentar el proyecto para que otra persona pueda instalar la aplicación, comprender el flujo experimental y reconocer sus limitaciones.

No se establece como objetivo alcanzar una exactitud concreta ni demostrar utilidad diagnóstica.

3. Conceptos teóricos

En este capítulo se explican los conceptos necesarios para entender el procesamiento de las señales, los modelos utilizados y la forma de evaluar sus resultados. No se pretende describir toda la teoría del EEG o del aprendizaje automático, sino los aspectos que intervienen en el proyecto.

3.1. Señales EEG y organización de los datos

La electroencefalografía registra diferencias de potencial mediante electrodos colocados sobre el cuero cabelludo. Los registros utilizados en este proyecto contienen 19 canales distribuidos según el sistema internacional 10–20 y fueron adquiridos a una frecuencia de muestreo de 128 Hz [17]. Esto significa que un segundo contiene 128 muestras por canal. El sistema 10–20 establece las posiciones de los electrodos, pero no fija una frecuencia de muestreo ni un número único de canales para todos los registros EEG.

En el CSV utilizado por la aplicación, cada fila representa una muestra temporal y cada canal se almacena en una columna. Los datos de entrenamiento incluyen además el identificador del sujeto, `ID`, y su clase, `Class`. Estos dos campos no se utilizan como características de entrada: el primero permite separar los sujetos durante la evaluación y la segunda es la etiqueta que el modelo debe aprender a predecir.

La aplicación comprueba que estén presentes los canales esperados y que sus columnas sean numéricas. Para el entrenamiento también exige los campos `ID` y `Class`. En la predicción no es necesario conocer la clase del registro, ya que es el resultado que se quiere obtener.

Esta organización limita los registros que se pueden utilizar directamente. Si un fichero emplea otros nombres para los canales o una disposición distinta de las columnas, debe adaptarse al formato esperado. Además, la frecuencia de muestreo no se deduce automáticamente del CSV: el procesamiento utiliza la frecuencia configurada, que en este trabajo es de 128 Hz. Un registro adquirido a otra frecuencia requiere adaptar previamente los datos o revisar la configuración del procesamiento y del modelo. Que un fichero cumpla el formato de entrada no garantiza que el modelo mantenga sus resultados con datos obtenidos en otras condiciones.

La Figura 3.1 muestra las primeras 640 muestras de tres canales del sujeto **v10p** en el CSV utilizado. El tiempo se calcula a partir del orden de las muestras y de la frecuencia de 128 Hz indicada en la descripción del dataset [8].

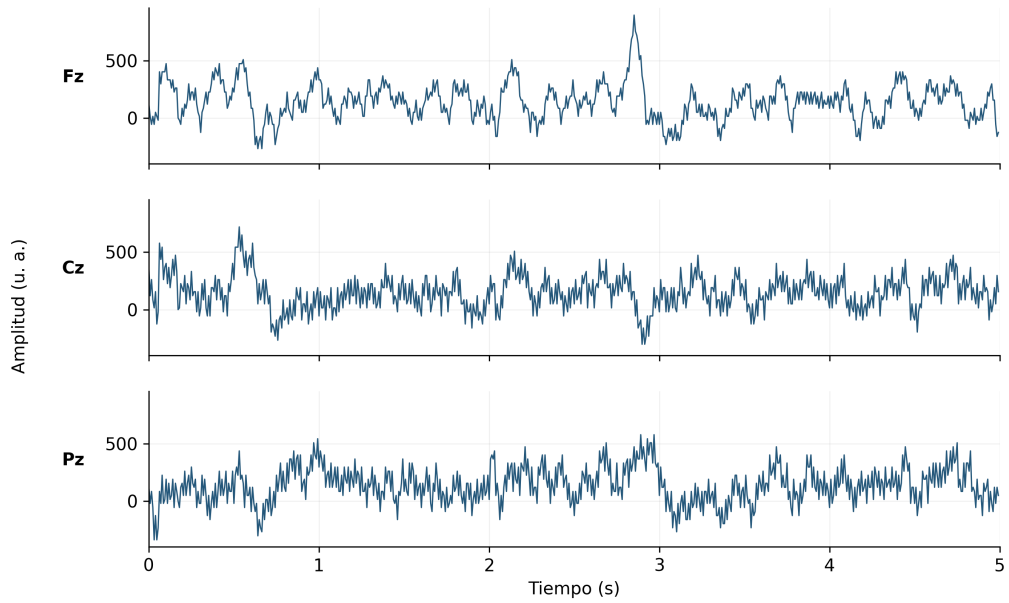


Figura 3.1: Primeras 640 muestras de los canales Fz, Cz y Pz del sujeto **v10p**, equivalentes a cinco segundos a 128 Hz. Amplitud en unidades arbitrarias, conservando los valores del CSV sin conversión a una unidad física ni filtrado o normalización adicional. Elaboración propia a partir del dataset utilizado.

3.2. Segmentación en ventanas

Los registros se dividen en segmentos de longitud fija, denominados ventanas o *epochs*. Esta representación permite aplicar el mismo modelo a registros de distinta duración. Hay que distinguir estas ventanas EEG de las épocas de entrenamiento de una red neuronal, que son recorridos sobre los datos de entrenamiento.

La segmentación depende del tamaño de ventana L y del desplazamiento S . Si S es menor que L , las ventanas consecutivas se solapan. Para un registro de N muestras, con $N \geq L$, el número de ventanas completas es:

$$n = 1 + \left\lfloor \frac{N - L}{S} \right\rfloor. \quad (3.1)$$

En el proyecto las ventanas se generan dentro del registro de cada sujeto, sin unir el final de un paciente con el comienzo del siguiente. El solape aumenta el número de ejemplos, pero no el número de pacientes independientes.

La Figura 3.2 ilustra este proceso con un ejemplo de 3840 muestras, equivalentes a 30 segundos a 128 Hz. Con ventanas de 1920 muestras y un desplazamiento de 960, se obtienen tres ventanas completas con un solape del 50 %. Cada ventana contiene el mismo intervalo temporal de los 19 canales.

3.3. Características temporales y espectrales

Características temporales

Los modelos clásicos reciben una tabla de características, donde cada fila representa una ventana. Para describir cada canal se calculan doce medidas temporales, que resumen distintos aspectos de la amplitud de la señal. La Tabla 3.1 recoge estas medidas.

Por ejemplo, para una ventana de un canal con muestras $x_1, \dots, x_L$, el valor RMS se calcula como:

$$\text{RMS}(x) = \sqrt{\frac{1}{L} \sum_{i=1}^L x_i^2}. \quad (3.2)$$

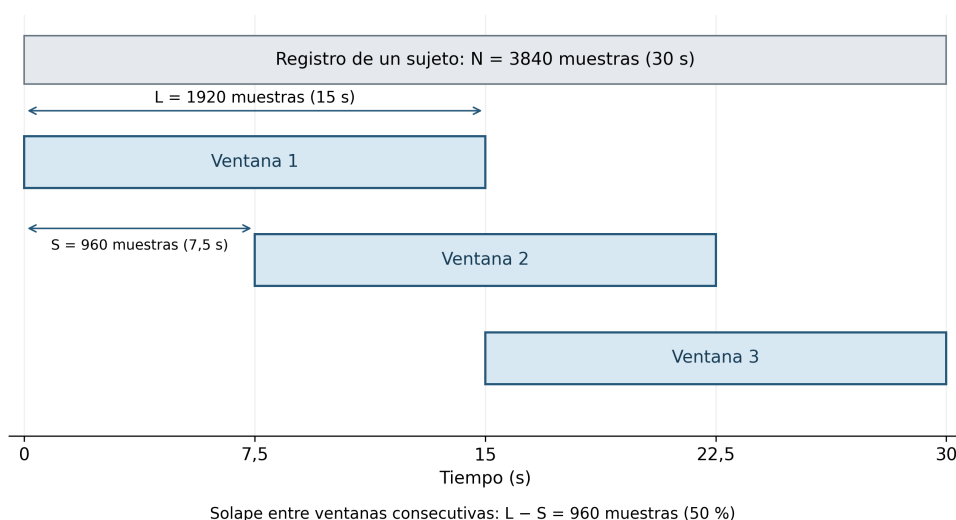


Figura 3.2: Ejemplo de segmentación de un registro de un sujeto en tres ventanas solapadas. L indica la longitud de cada ventana y S el desplazamiento entre sus inicios. Esquema de elaboración propia.

Grupo	Medidas por canal
Tendencia central	Media y mediana
Dispersión	Desviación típica, varianza, rango y rango intercuartílico
Extremos y percentiles	Mínimo, máximo y percentiles 25 y 75
Energía y amplitud	Energía y valor RMS

Tabla 3.1: Las doce características temporales calculadas por canal y ventana. Elaboración propia a partir de la implementación.

Al calcular estas doce medidas para cada uno de los 19 canales, se obtienen $12 \times 19 = 228$ características temporales por ventana. Por tanto, cada ventana queda representada por un vector de 228 valores, además de la etiqueta y el identificador del sujeto, que se mantienen separados de las características de entrada.

Características espectrales

La información espectral describe cómo se distribuye la potencia de la señal en frecuencia. Para estimar la densidad espectral de potencia se utiliza el método de Welch, que divide la señal en segmentos, calcula sus

periodogramas modificados y los promedia [40]. En el proyecto se emplea la implementación disponible en la función `scipy.signal.welch` [35].

En el código se utilizan las bandas delta (0,5–4 Hz), theta (4–8 Hz), alfa (8–13 Hz), beta (13–30 Hz) y gamma (30–45 Hz). Estos intervalos son la configuración del proyecto y no se presentan como una definición única para todos los estudios EEG.

La potencia absoluta por banda resume la potencia contenida en un intervalo de frecuencias. La potencia relativa expresa su proporción respecto a la potencia total del intervalo analizado. Estas medidas permiten representar parte del contenido frecuencial de la señal mediante variables numéricas [7].

Además de las potencias absolutas y relativas, se calculan la entropía espectral y el cociente theta/beta en cada canal, y la frecuencia media de la banda beta en O1 y O2. En total se obtienen 230 características espectrales por ventana. La Tabla 3.2 resume su distribución.

Característica	Cantidad por ventana
Potencia absoluta en cinco bandas y 19 canales	95
Potencia relativa en cinco bandas y 19 canales	95
Entropía espectral en 19 canales	19
Cociente theta/beta en 19 canales	19
Frecuencia media de la banda beta en O1 y O2	2
Total	230

Tabla 3.2: Distribución de las características espectrales por ventana. Elaboración propia a partir de la implementación.

Al combinar las 228 características temporales con las 230 espectrales, cada ventana queda representada por un vector de 458 valores. Este es el tamaño de entrada de los modelos clásicos cuando se utiliza la configuración de características combinadas. La presencia de una característica en este conjunto no implica que por sí sola permita identificar TDAH.

3.4. Modelos de Machine Learning

Se utilizan varios clasificadores para comparar enfoques distintos. La regresión logística establece una referencia lineal; KNN clasifica a partir de ejemplos cercanos; SVM con kernel RBF permite establecer una separación no lineal; y Random Forest combina árboles de decisión. Estos métodos están descritos en la documentación de aprendizaje supervisado de scikit-learn [34].

También se utiliza XGBoost, basado en conjuntos de árboles contruidos mediante boosting [4]. En lugar de asumir que una familia será mejor, se compara su comportamiento en las particiones definidas para el proyecto.

El escalado de las características forma parte del pipeline de regresión logística, SVM y KNN. Los parámetros del escalador se calculan únicamente con los datos de entrenamiento de cada fold y se aplican después a los datos de prueba. Los pipelines de Random Forest y XGBoost utilizados no incorporan ese escalador.

3.5. Modelos de Deep Learning

Las redes neuronales reciben ventanas con dimensiones tiempo por canales. A diferencia del enfoque clásico, no se les proporciona la tabla de estadísticas temporales y espectrales calculada manualmente.

La CNN 1D aplica filtros convolucionales que recorren la dimensión temporal de la ventana para aprender patrones locales de la señal [19]. La CNN-LSTM combina bloques convolucionales con una capa LSTM bidireccional, que procesa la secuencia resultante en ambos sentidos para incorporar información sobre sus dependencias temporales [20, 18].

Se evalúa también una variante ligera inspirada en EEGNet, cuya propuesta original utiliza convoluciones específicas para trabajar con EEG [23]. La implementación del proyecto es una adaptación con capas unidimensionales, no una reproducción exacta de la arquitectura del artículo.

Durante el entrenamiento se utilizan mecanismos como dropout y parada temprana. Su finalidad es controlar el ajuste del modelo; no garantizan por sí solos que desaparezca el sobreajuste.

3.6. Validación por paciente

La validación cruzada divide los datos en varios grupos o folds. En cada iteración se entrena con una parte y se evalúa con otra. Cuando existen varias muestras de un mismo sujeto, la separación debe respetar esa agrupación [31].

En el proyecto se utiliza `StratifiedGroupKFold` con cinco folds. Se intenta mantener la proporción de clases sin repartir las ventanas de un mismo paciente entre entrenamiento y prueba dentro de una misma iteración. Así, el modelo se evalúa sobre sujetos que no han participado en su entrenamiento.

En DL se reserva además aproximadamente el 20 % de los sujetos de la parte de entrenamiento para una validación interna. Esta se utiliza para la parada temprana y la selección del umbral de decisión. Los sujetos del fold de prueba quedan fuera de estos ajustes.

Las predicciones *out-of-fold* se obtienen sobre las ventanas de prueba de cada iteración. Al reunir las, cada ventana dispone de una predicción realizada por un modelo que no se ha entrenado con su sujeto. Permiten construir una figura conjunta, aunque proceden de modelos ajustados en distintos folds.

3.7. Métricas de evaluación

La exactitud indica la proporción de predicciones correctas. La exactitud balanceada promedia el recall de las clases, mientras que precisión y recall describen, respectivamente, los aciertos entre las predicciones de una clase y la parte de esa clase que se consigue recuperar. F1 combina precisión y recall [32].

Para una clase, tomando TP , FP y FN como verdaderos positivos, falsos positivos y falsos negativos:

$$P = \frac{TP}{TP + FP}, \quad R = \frac{TP}{TP + FN}, \quad F1 = \frac{2PR}{P + R}. \quad (3.3)$$

El código calcula precisión, recall y F1 con promedio ponderado por el número de ejemplos de cada clase. Por tanto, el recall global de las tablas no debe interpretarse directamente como sensibilidad específica para TDAH. Las curvas ROC muestran la relación entre verdaderos y falsos positivos al variar el umbral; su área se resume mediante AUC.

Las tablas experimentales de esta memoria presentan métricas por ventana. La aplicación también agrega los votos de las ventanas para obtener una decisión por paciente. Son dos niveles diferentes de evaluación y no deben confundirse.

3.8. Importancia de características

La importancia por permutación mide cómo cambia una métrica cuando se desordena una característica manteniendo el resto de los datos. Una caída mayor indica que el modelo dependía más de esa información en el conjunto evaluado [33].

Las repeticiones reducen la dependencia de una única permutación, pero no sustituyen repetir la evaluación con otros pacientes. Las características correlacionadas pueden repartirse la importancia y también pueden aparecer valores negativos. La suma por canal es una forma de resumir las características, no una medida de causalidad ni una localización clínica del trastorno.

4. Técnicas y herramientas

En este capítulo se recogen las técnicas y herramientas utilizadas y el papel que tienen dentro del proyecto. La parte experimental y la aplicación comparten parte del código de procesamiento, pero tienen necesidades distintas: en los scripts interesa poder repetir un experimento y generar sus figuras, mientras que en la aplicación también hay que gestionar usuarios, ficheros y trabajos en segundo plano.

4.1. Técnicas de desarrollo y evaluación

El desarrollo se ha realizado de forma incremental, incorporando funcionalidades sobre versiones que ya podían ejecutarse. Las tareas se han organizado mediante issues y un tablero de seguimiento. La planificación y la evolución del proyecto se describen en el Anexo A.

En el diseño de la aplicación se aplica una separación de responsabilidades entre la interfaz, la API, la lógica de procesamiento y la persistencia. Dentro del backend se distinguen las rutas que reciben las peticiones, los servicios que realizan las operaciones y los repositorios que acceden a la base de datos. La estructura y sus diagramas se detallan en el Anexo C.

El análisis de datasets y los entrenamientos se ejecutan de forma asíncrona mediante una cola de tareas. La API registra el trabajo y permite consultar su estado mientras un proceso independiente realiza el cálculo. Esto permite seguir utilizando la interfaz durante su ejecución.

En la parte experimental se utiliza validación cruzada por sujeto, manteniendo separadas las ventanas de los pacientes de entrenamiento y prueba en cada fold. El funcionamiento de esta evaluación se explica en el capítulo

de conceptos teóricos. Las pruebas automatizadas y las comprobaciones de integración continua completan el proceso de desarrollo.

4.2. Procesamiento de datos y entrenamiento

Python se utiliza tanto en los scripts de investigación como en el backend. NumPy permite trabajar con las señales y las ventanas como matrices, y pandas se utiliza para cargar los CSV y organizar los resultados. El procesamiento de señales se apoya en SciPy, por ejemplo para el filtrado y para calcular la densidad espectral mediante Welch [35].

Los modelos de aprendizaje automático se implementan con scikit-learn y XGBoost. Scikit-learn permite mantener el escalado y el clasificador dentro de un mismo pipeline, además de proporcionar las particiones por grupos y las métricas de evaluación [31, 32]. XGBoost se incorpora como otra alternativa basada en árboles [4].

Para las redes neuronales se utiliza TensorFlow con Keras. Esta combinación permite definir las arquitecturas mediante capas y utilizar funciones integradas para el entrenamiento, la parada temprana y el guardado de modelos [22, 21]. En el proyecto, estas funciones facilitan mantener un procedimiento común para las distintas redes y recuperar los modelos entrenados para realizar predicciones. No se ha realizado una comparación experimental con PyTorch, por lo que la elección no se presenta como una ventaja de rendimiento frente a esta alternativa.

Matplotlib se utiliza para representar los resultados de los experimentos.

4.3. Backend y persistencia

La API está desarrollada con FastAPI. Los esquemas de Pydantic definen los datos de entrada y salida, y la documentación interactiva facilita comprobar las peticiones durante el desarrollo [13]. El código está organizado por dominios funcionales para que la carga de datasets, el entrenamiento o la autenticación no dependan de un único fichero con toda la lógica.

PostgreSQL almacena los usuarios, experimentos y metadatos de los recursos [28]. SQLAlchemy se utiliza para acceder a la base de datos y Alembic para versionar los cambios. Los ficheros de modelos y datasets se

conservan en disco; la base de datos mantiene la información necesaria para localizarlos y relacionarlos con sus propietarios.

Los trabajos largos se ejecutan con Celery y Redis. Celery permite separar su ejecución del proceso que atiende las peticiones HTTP [3]. Redis actúa como intermediario de la cola y como almacén temporal de estados y resultados. Esta separación es especialmente útil en el entrenamiento, donde el cálculo puede durar bastante más que una petición HTTP habitual.

4.4. Interfaz web

La interfaz está desarrollada con React y TypeScript, utilizando Vite como herramienta de desarrollo y construcción [24, 39]. React permite dividir las pantallas en componentes, mientras que TypeScript ayuda a detectar incompatibilidades de tipos durante el desarrollo.

Las pantallas, componentes y hooks se agrupan por funcionalidad. Las llamadas a la API utilizan código común para no repetir la gestión de peticiones en cada vista. La página de presentación es un proyecto estático independiente, también construido con Vite, que presenta el trabajo y cuenta con un enlace a la aplicación.

4.5. Entorno y despliegue

Las dependencias de Python se declaran en `pyproject.toml` y se fijan mediante el archivo de bloqueo de uv [1]. La configuración actual utiliza Python 3.13. Para el frontend se mantienen las dependencias de npm junto con su archivo de bloqueo. Esto reduce las diferencias entre los entornos, aunque reproducir un resultado de entrenamiento también depende de los datos, la configuración y las bibliotecas empleadas.

Docker Compose reúne los servicios necesarios para ejecutar la aplicación [9]. En el despliegue, Caddy dirige las peticiones hacia la interfaz o hacia la API, y Cloudflare Tunnel publica el servicio sin abrir directamente un puerto de entrada en el router [6]. El cálculo sigue realizándose en el equipo que ejecuta los contenedores. La página de presentación se despliega por separado en Cloudflare Pages [5].

4.6. Control de versiones y calidad

Git y GitHub se utilizan para mantener el historial de cambios y revisar las ramas mediante pull requests [36, 16]. Las tareas se organizan con issues y un tablero de ZenHub [41]. El seguimiento y las etapas del trabajo se desarrollan con más detalle en el anexo de planificación.

GitHub Actions ejecuta las comprobaciones automáticas [15]. En Python se emplean Ruff y pytest, junto con el informe de cobertura; en el frontend se comprueban los tipos, el lint y la construcción. SonarQube Cloud complementa estas comprobaciones con análisis estático [29, 30, 37]. Estas herramientas permiten detectar errores antes de integrar cambios.

4.7. Uso de inteligencia artificial generativa

Durante el desarrollo del proyecto se ha utilizado Codex, de OpenAI, como herramienta de apoyo para la implementación y reorganización de partes del código, la resolución de errores y la preparación de la configuración necesaria para el despliegue de la aplicación mediante Cloudflare. También se ha empleado como apoyo en la revisión de la documentación, la detección de posibles incoherencias y la preparación de figuras explicativas mediante scripts.

Este apoyo ha incluido propuestas de código y de redacción. La responsabilidad sobre su incorporación, la comprobación del funcionamiento y la justificación de las decisiones adoptadas corresponde al autor del trabajo.

5. Aspectos relevantes del desarrollo del proyecto

El trabajo tiene dos partes relacionadas. Primero se preparó el proceso experimental para comparar modelos sobre señales EEG. Después se construyó una aplicación que permite utilizar ese proceso sin ejecutar ni modificar manualmente los scripts. En este capítulo se explican las decisiones que afectan a los resultados y los principales cambios realizados durante el desarrollo de la aplicación.

5.1. Preparación de los datos

El conjunto utilizado procede de *EEG data for ADHD / Control children*, publicado en IEEE DataPort, y contiene registros EEG de niños de Irán [10]. En el proyecto se ha trabajado con la copia disponible en Kaggle [8], por lo que se distingue entre el origen del conjunto y la distribución utilizada. La configuración de los experimentos guardados recoge 121 sujetos. El código trabaja con 19 canales EEG y una frecuencia de muestreo de 128 Hz. Cada registro se identifica mediante el sujeto al que pertenece y su clase, ADHD o Control.

Los datos pertenecen a una población infantil concreta. La evaluación mantiene separados los sujetos de entrenamiento y prueba, pero todos proceden del mismo conjunto de datos. Por ello, los resultados no permiten afirmar que el modelo vaya a funcionar igual con adultos, otras poblaciones o registros obtenidos en condiciones distintas.

Para comprobarlo sería necesario evaluar los modelos con un conjunto independiente, revisando la compatibilidad de los canales, la frecuencia de

muestreo, la referencia de los electrodos y las etiquetas utilizadas. Otra copia del mismo dataset no serviría como evaluación externa. Esta comprobación queda como trabajo futuro.

Los identificadores y la clase se separan de las columnas de señal antes de generar las entradas de los modelos. El identificador se conserva para construir las particiones, pero no se utiliza como una característica de clasificación. Si el modelo recibiera este dato, podría aprender relaciones que no representan el comportamiento de la señal.

Se preparan ventanas de longitud fija para poder entrenar con registros de distinta duración. Las ventanas pueden solaparse dentro de un mismo registro, pero todas las de un sujeto deben quedar en el mismo lado de cada partición. Por tanto, el número de ventanas no debe interpretarse como el número de pacientes independientes del estudio.

5.2. Diseño de los experimentos

Aprendizaje automático

La configuración guardada para ML utiliza ventanas de 1920 muestras y un paso de 960, equivalentes a 15 y 7,5 segundos, respectivamente. Con estos parámetros se obtienen 2079 ventanas. Se combinan características temporales y espectrales hasta formar 458 variables por ventana. La longitud y el paso de las ventanas, así como los hiperparámetros de los clasificadores, se eligieron a partir de las pruebas exploratorias realizadas en el notebook de aprendizaje automático.

En esta configuración no se aplica el filtrado ni la normalización por sujeto de la rama DL. Esto no significa que todos los clasificadores reciban las variables sin escalar: los pipelines de regresión logística, SVM y KNN incluyen `StandardScaler`, ajustado solamente con el entrenamiento de cada partición. Random Forest y XGBoost no utilizan este paso.

Se comparan regresión logística, KNN, SVM con kernel RBF, Random Forest y XGBoost. La evaluación se realiza con cinco particiones mediante `StratifiedGroupKFold`, manteniendo los grupos por sujeto [31]. En cada una se crea y ajusta un modelo nuevo. Las métricas finales son la media y la desviación típica de las cinco evaluaciones.

El script selecciona el modelo con mayor F1 ponderado medio. Por tanto, cuando se habla del mejor modelo ML en estos resultados, se está haciendo

referencia a este criterio y a esta configuración, no a una superioridad general frente a cualquier otro modelo.

Aprendizaje profundo

En DL se utilizan ventanas de 512 muestras y un paso de 256, equivalentes a 4 y 2 segundos. La configuración guardada contiene 8279 ventanas, cada una con 512 muestras y 19 canales. En este caso se aplica un filtro paso banda de 0,5 a 50 Hz y normalización por sujeto y canal.

El filtrado y la normalización se realizan sobre el registro del propio sujeto. No se mezclan registros de pacientes de entrenamiento y prueba para calcularlos. Sin embargo, al utilizar el registro completo y un filtro no causal, este procesamiento corresponde a un análisis de ficheros y no a una predicción en tiempo real muestra a muestra.

Se comparan CNN 1D, CNN-LSTM y una arquitectura ligera inspirada en EEGNet. También se utilizan cinco particiones externas por sujeto. Dentro del entrenamiento de cada partición se reserva un 20 % de los sujetos para validación. Esta parte se utiliza para la parada temprana y la elección del umbral, sin recurrir a los sujetos de prueba.

La configuración contempla un máximo de 40 épocas de entrenamiento, lotes de 32 y parada temprana cuando la pérdida de validación deja de mejorar. El umbral se busca entre 0,2 y 0,8, priorizando la exactitud balanceada en validación. Estos pasos reducen la dependencia de un umbral fijo, pero también forman parte del procedimiento que habría que mantener al reproducir el experimento.

Comparación de las configuraciones

Las dos ramas comparten la separación por sujeto, pero utilizan distintas formas de preparar las entradas. La tabla 5.1 reúne las principales diferencias de los experimentos descritos.

Aspecto	Machine Learning	Deep Learning
Longitud de ventana	1920 muestras, 15 segundos	512 muestras, 4 segundos
Desplazamiento	960 muestras, 7,5 segundos	256 muestras, 2 segundos
Ventanas obtenidas	2079	8279
Entrada del modelo	458 características por ventana	Matriz de 512 muestras y 19 canales
Filtrado	Sin filtrado adicional en esta configuración	Paso banda de 0,5 a 50 Hz
Escalado o normalización	Escalado en los pipelines de regresión logística, SVM y KNN	Normalización por sujeto y canal
Evaluación	Cinco particiones por sujeto	Cinco particiones externas por sujeto
Validación interna	Sin una partición interna equivalente a la de DL	Reserva del 20 % de los sujetos de entrenamiento

Tabla 5.1: Configuraciones utilizadas en los experimentos ML y DL.

En ML se calculan características temporales y espectrales a partir de ventanas de 15 segundos. En DL, la red recibe la señal preprocesada de los 19 canales en ventanas de 4 segundos y aprende patrones durante el entrenamiento. Por ello, las diferencias de rendimiento pueden depender tanto del modelo como de la preparación de los datos.

El mayor número de ventanas de DL no supone disponer de más sujetos independientes. Ambas ramas parten de los mismos 121 sujetos; el cambio en la longitud y el desplazamiento produce un número distinto de segmentos. La separación por sujeto se mantiene en las dos evaluaciones.

El objetivo ha sido buscar una configuración adecuada para cada familia de modelos a partir de las pruebas realizadas. Por este motivo, se han utilizado distintas preparaciones de los datos en ML y DL. Los resultados corresponden a esas configuraciones completas y no permiten atribuir las diferencias únicamente al tipo de modelo.

5.3. Resultados de clasificación

Las tablas 5.2 y 5.3 recogen los resultados guardados en los archivos `ml_cv_summary.csv` y `dl_cv_summary.csv`. Se muestran porcentajes como

media y desviación típica entre particiones. El F1 es ponderado por el soporte de cada clase y todas estas métricas se calculan por ventana.

Modelo	Exactitud	Ex. balanceada	F1 ponderado
Regresión logística	68,85 $\pm$ 7,56	68,66 $\pm$ 8,39	68,78 $\pm$ 7,79
KNN	73,73 $\pm$ 5,78	72,75 $\pm$ 6,11	73,38 $\pm$ 5,89
SVM RBF	75,37 $\pm$ 4,23	74,89 $\pm$ 4,67	75,20 $\pm$ 4,48
Random Forest	78,86 $\pm$ 5,82	78,25 $\pm$ 5,58	78,72 $\pm$ 5,78
XGBoost	77,51 $\pm$ 5,62	77,22 $\pm$ 5,13	77,46 $\pm$ 5,54

Tabla 5.2: Resultados ML por ventana en cinco particiones por sujeto. Media y desviación típica expresadas en porcentaje.

Random Forest obtiene el mayor F1 ponderado medio, con un 78,72 %, seguido de XGBoost, con un 77,46 %. La diferencia es pequeña en relación con la variación entre particiones. No se ha realizado una prueba estadística que permita afirmar que esa diferencia sea significativa.

Modelo	Exactitud	Ex. balanceada	F1 ponderado
CNN 1D	80,37 $\pm$ 7,59	80,46 $\pm$ 7,56	80,14 $\pm$ 7,76
CNN-LSTM	79,01 $\pm$ 2,15	78,63 $\pm$ 2,40	78,64 $\pm$ 2,28
Inspirado en EEGNet	74,35 $\pm$ 13,36	74,08 $\pm$ 15,25	71,41 $\pm$ 18,35

Tabla 5.3: Resultados DL por ventana en cinco particiones por sujeto. Media y desviación típica expresadas en porcentaje.

CNN 1D obtiene el mayor F1 ponderado medio entre las redes evaluadas, con un 80,14 %. CNN-LSTM presenta una menor desviación entre particiones en este experimento. La adaptación de EEGNet muestra una variabilidad mayor, por lo que su media por sí sola no describe bien lo ocurrido en las distintas particiones.

La CNN 1D también tiene un F1 medio superior al de Random Forest en los archivos guardados, pero no es una comparación controlada únicamente de la arquitectura. ML y DL utilizan ventanas, preprocesamientos y representaciones diferentes. Además, un sujeto con un registro más largo puede aportar más ventanas a las métricas. Para comparar decisiones por paciente sería necesario agregar las predicciones y volver a calcular la evaluación a ese nivel.

Discusión de los resultados

En los modelos de Machine Learning, Random Forest y XGBoost obtienen los mayores valores medios de F1 ponderado. La regresión logística presenta los resultados más bajos, mientras que KNN y SVM se sitúan en una posición intermedia. Para las características y la configuración utilizadas, los modelos basados en árboles han funcionado mejor según esta métrica. Aun así, las métricas globales no permiten determinar qué características concretas explican esa diferencia.

En Deep Learning, CNN 1D obtiene el mayor F1 ponderado medio, con un 80,14 %, frente al 78,64 % de CNN-LSTM. La diferencia entre ambas medias es de 1,50 puntos porcentuales. CNN-LSTM presenta una menor variación entre las cinco particiones evaluadas: su desviación típica es de 2,28 puntos, frente a los 7,76 de CNN 1D.

La arquitectura inspirada en EEGNet obtiene un F1 ponderado medio del 71,41 %, con una desviación típica de 18,35 puntos. Esto muestra que su rendimiento cambia considerablemente entre particiones. Sería necesario analizar cada partición por separado para estudiar las causas de esta variación.

En los scripts, los modelos se seleccionan según el F1 ponderado medio. Por este motivo, Random Forest es el modelo seleccionado dentro de Machine Learning y CNN 1D dentro de Deep Learning. La variación entre particiones aporta información adicional sobre cómo cambia el rendimiento al repetir la evaluación con otros grupos de sujetos.

Curvas ROC

La figura 5.1 reúne las curvas de Random Forest, SVM RBF y XGBoost. Se construye con las puntuaciones de prueba obtenidas fuera del entrenamiento de cada partición, también llamadas predicciones *out-of-fold*.

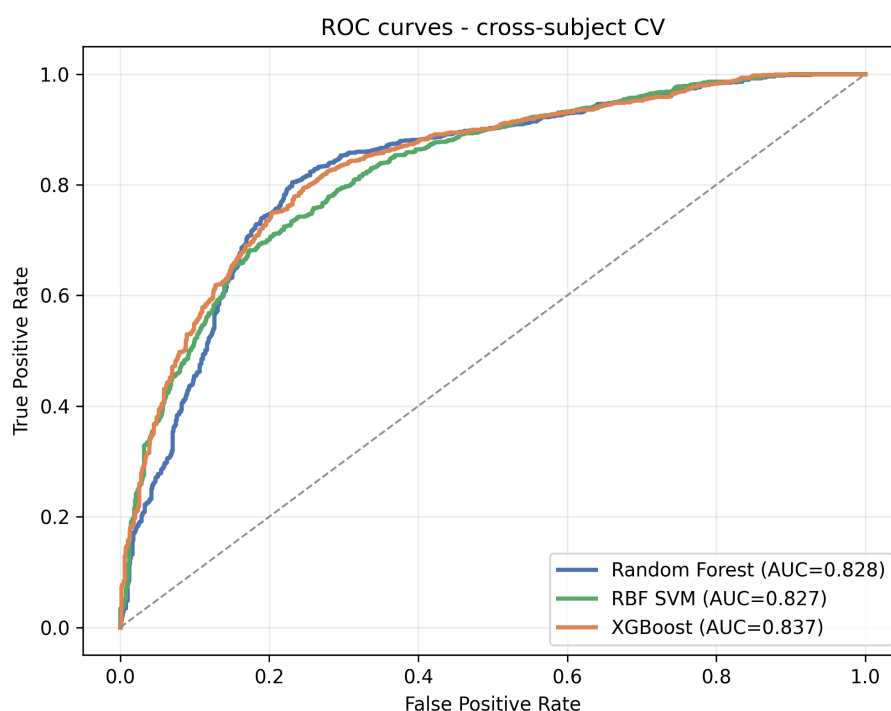


Figura 5.1: Curvas ROC de los tres clasificadores ML a partir de las predicciones de prueba de la validación cruzada por sujeto.

En esta figura XGBoost obtiene un AUC de 0,837, Random Forest de 0,828 y SVM de 0,827. Esto no contradice la selección de Random Forest: el AUC valora el orden de las puntuaciones a lo largo de distintos umbrales, mientras que la selección del script utiliza el F1 ponderado medio de las particiones.

La curva reúne predicciones de los modelos ajustados en cada partición. Por tanto, no equivale a evaluar un único modelo final sobre otro conjunto de datos independiente. Esta distinción es importante al describir la figura y al compararla con otros trabajos.

Importancia de características

Para estudiar qué variables utiliza el modelo se calcula importancia por permutación, midiendo el cambio del F1 ponderado al alterar cada característica [33]. El análisis de los scripts utiliza una separación por sujeto con un 20 % reservado para prueba y diez repeticiones de las permutaciones. Se vuelve a ajustar el estimador con el entrenamiento de esa separación.

Las diez repeticiones no son diez entrenamientos independientes. Sirven para reducir la variación del cálculo debida al orden aleatorio de las permutaciones. Además del resultado por característica, se suman sus importancias por canal EEG para facilitar la lectura.

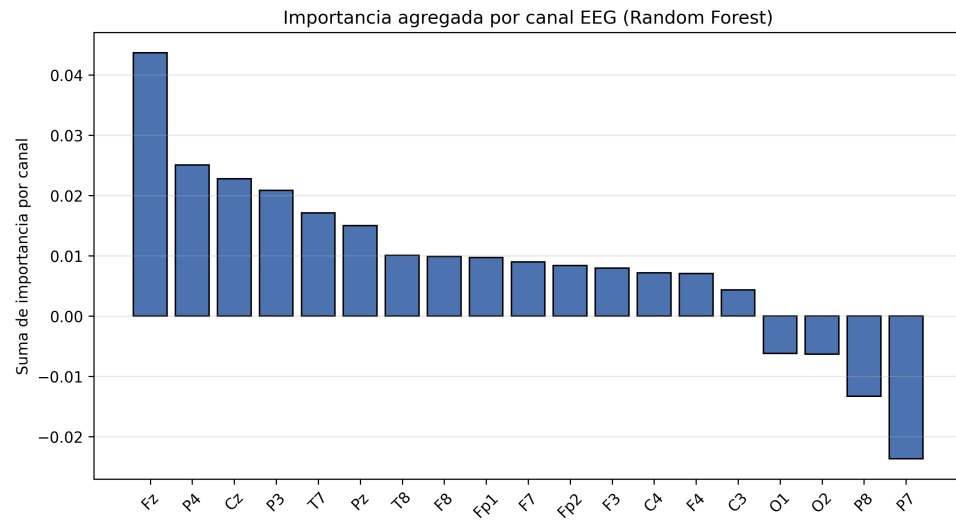


Figura 5.2: Importancia por permutación agregada por canal para Random Forest.

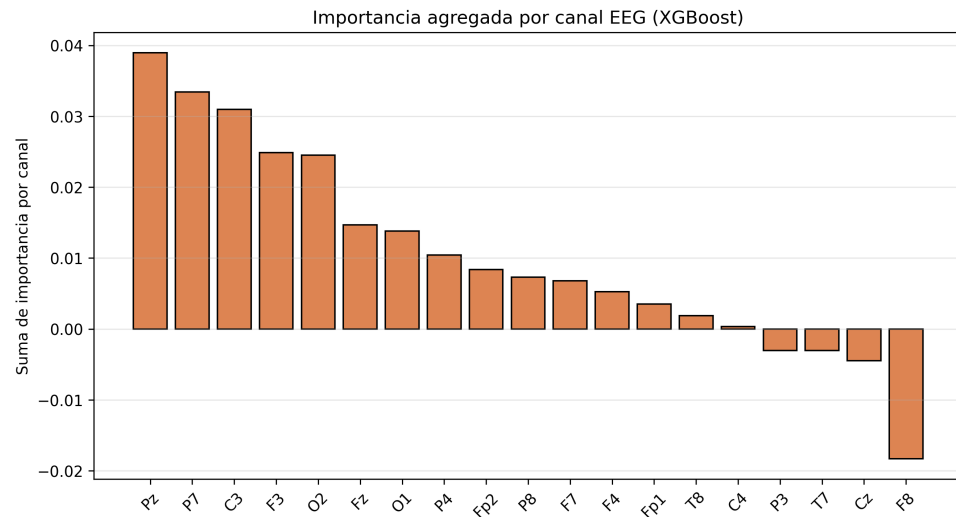


Figura 5.3: Importancia por permutación agregada por canal para XGBoost.

Fz y Pz identifican posiciones de electrodos en el sistema 10–20. La letra F corresponde a la región frontal, la P a la parietal y la z indica que el electrodo está situado en la línea media de la cabeza [25]. En estas figuras, la importancia de cada canal se obtiene sumando las importancias de las características calculadas a partir de su señal.

Los canales destacados no coinciden completamente entre ambos modelos. En las figuras guardadas, Fz presenta la mayor importancia agregada en Random Forest, mientras que Pz ocupa esa posición en XGBoost. La contribución depende del modelo, de la partición y de las relaciones entre variables.

Una importancia negativa indica que la permutación ha mejorado la métrica en ese cálculo. No significa que el canal proteja frente al TDAH ni que exista una relación causal. Tampoco debe entenderse la suma por canal como una medida clínica: agrupa perturbaciones individuales de características que pueden estar correlacionadas.

Esta partición de análisis procede del mismo conjunto de datos utilizado en el estudio. Aunque los sujetos de prueba no se usan para ajustar ese estimador, no constituye una validación externa independiente de las decisiones tomadas durante el proyecto.

5.4. Del experimento a la aplicación

Del prototipo a React y FastAPI

La primera interfaz se desarrolló con Streamlit, un framework de Python que permite crear aplicaciones web interactivas orientadas a datos [38]. Se utilizó porque permitía integrar los modelos y visualizar sus resultados de forma sencilla. En esa etapa, el objetivo era disponer de un prototipo ligado al trabajo experimental de aprendizaje automático y ciencia de datos.

Al ampliar el proyecto se buscó una aplicación más completa, con más funcionalidades y un flujo que permitiera continuar después de realizar una predicción. Se quería poder cargar datasets, entrenar, guardar experimentos y reutilizar los modelos desde la propia interfaz. Al observar las limitaciones del prototipo para la organización y el flujo que se buscaban, se decidió separar el frontend en React y el backend en FastAPI.

No se trataba de que Streamlit impidiera añadir funcionalidades, sino de disponer de más control sobre la interfaz y la organización de los distintos flujos de uso. Esta decisión también supuso más trabajo: definir la comuni-

cación con la API, gestionar los errores y mantener ambas partes. Sobre esa base se incorporaron después los usuarios y los trabajos en segundo plano.

Configuración desde la interfaz

La aplicación permite seleccionar el dataset, la familia ML o DL y el modelo que se quiere entrenar. También permite modificar la longitud y el desplazamiento de las ventanas, activar el preprocesamiento de filtrado y normalización, y ajustar los hiperparámetros disponibles para cada modelo.

En ML se puede elegir entre características temporales, espectrales o combinadas, además del tamaño de segmento utilizado por Welch. En DL se configuran el máximo de épocas de entrenamiento, el tamaño de lote, la tasa de aprendizaje y la paciencia de la parada temprana. Las opciones se ofrecen mediante valores predefinidos; la frecuencia de muestreo disponible es de 128 Hz.

Estas opciones permiten realizar nuevos experimentos desde la interfaz. Los resultados presentados en esta memoria corresponden a las configuraciones descritas anteriormente. Durante la inferencia se recuperan los parámetros asociados al modelo seleccionado para preparar los datos de forma coherente con su entrenamiento.

Registro de modelos e inferencia

Una de las partes que hubo que completar fue la conexión entre entrenar y utilizar después el modelo. No bastaba con guardar las métricas del experimento: también había que exportar el modelo final, conservar su configuración y mostrarlo en el selector de inferencia.

La aplicación registra los metadatos del modelo y su relación con el experimento en PostgreSQL, mientras que el fichero del modelo se guarda en disco. La inferencia necesita tanto el modelo como los parámetros de preparación de sus entradas. Aplicar otra longitud de ventana o un conjunto distinto de características podría provocar un error o producir resultados que no corresponden al entrenamiento.

La preparación de los datos también forma parte del uso del modelo. Por ejemplo, si un clasificador se ha entrenado con 458 características, durante la inferencia debe recibir esas mismas características y en el mismo orden. No basta con que el nuevo fichero contenga señales EEG: sus datos deben transformarse de acuerdo con la configuración utilizada en el entrenamiento.

Por este motivo, el flujo de inferencia combina la validación del CSV, la preparación de las ventanas y la ejecución del modelo seleccionado. Conservar la configuración permite repetir estos pasos sin que el usuario tenga que reconstruir manualmente el procesamiento aplicado al entrenar.

También se distinguen dos resultados del entrenamiento: la evaluación del experimento y el modelo exportado para su uso posterior. Las métricas permiten consultar cómo ha funcionado la configuración evaluada, mientras que el fichero del modelo permite realizar nuevas predicciones. Esta distinción explica por qué guardar un experimento y disponer de un modelo para inferencia son operaciones relacionadas, pero diferentes.

El selector permite utilizar modelos entrenados por el usuario y modelos de referencia compartidos.

Si la evaluación termina pero falla la exportación, el experimento puede conservar sus resultados aunque el modelo no esté disponible para inferencia. La aplicación debe mostrar esta situación sin presentar toda la ejecución como fallida.

El criterio de mejor modelo en la aplicación no debe confundirse con el de los scripts: la ordenación de la aplicación prioriza la exactitud balanceada y después el F1, mientras que los resultados experimentales anteriores seleccionan por F1 medio.

Trabajos en segundo plano

El análisis de datasets y los entrenamientos ML y DL se trasladaron a Celery. La API inicia el trabajo y devuelve un identificador, y la interfaz consulta su estado hasta recibir el resultado. De esta manera el usuario puede cambiar de pantalla sin mantener una petición de entrenamiento abierta.

Redis conserva temporalmente el estado y el resultado de la tarea. PostgreSQL mantiene su identificación y propietario, pero no es un historial permanente de todos los cambios de estado de Celery. La configuración actual limita el worker a una tarea simultánea, por lo que los trabajos adicionales esperan en la cola. Separar la ejecución de la API mejora la respuesta de la aplicación, pero no aumenta por sí solo la capacidad de cálculo del equipo.

Durante las comprobaciones del despliegue se detectaron interrupciones del worker por falta de memoria al ejecutar entrenamientos de Deep Learning. La generación de las particiones mantenía en una lista las copias de

entrenamiento y prueba de todos los pliegues. Se cambió por un iterador que produce una partición cada vez, evitando conservar simultáneamente todas esas copias. Este cambio mantiene la separación por sujeto y el procedimiento de validación cruzada.

También se revisó la memoria asignada a Docker sobre WSL2, que estaba limitada a unos 4 GB, y se amplió a 8 GB en el equipo utilizado. Después se completó un entrenamiento de comprobación con CNN 1D, cinco particiones y una época de entrenamiento por partición. Esta ejecución sirvió para comprobar el funcionamiento del flujo y el guardado del modelo; no sustituye los experimentos de clasificación presentados en este capítulo ni establece un requisito mínimo de memoria.

La interfaz conserva además el resumen de la tarea enviada para que la barra de estado muestre el modelo y el dataset que se están utilizando, aunque el usuario cambie la selección del formulario. De esta manera, la configuración de una nueva ejecución no se confunde con la del trabajo que ya está en curso.

Usuarios y organización del código

La incorporación de usuarios añade registro, inicio de sesión y consulta del perfil. Las contraseñas se almacenan mediante hash y las peticiones privadas requieren autenticación. Además de comprobar el token, el backend verifica el acceso al recurso solicitado. Ocultar un experimento en la interfaz no sería suficiente si otro usuario pudiera consultarlo cambiando su identificador.

El crecimiento de estas funcionalidades motivó la reorganización del backend por dominios y del frontend por funcionalidades. El objetivo era separar mejor las responsabilidades del código y facilitar su mantenimiento y futuras ampliaciones. La migración a TypeScript ayudó a definir los datos que utiliza la interfaz y a detectar errores en su uso. Esta organización facilita ampliar la aplicación sin acumular todo el código en unas pocas pantallas o servicios.

El diseño se detalla en el Anexo C. La Figura C.2 muestra los componentes de la aplicación y sus relaciones, mientras que las Figuras C.3 y C.4 recogen la organización del backend y del frontend. El despliegue y la secuencia de entrenamiento asíncrono se representan en las Figuras C.5 y C.6, respectivamente.

5.5. Publicación y comprobación del sistema

El despliegue diferencia la landing pública de la aplicación. La primera se sirve como contenido estático en Cloudflare Pages; la segunda llega al equipo que ejecuta Docker mediante Cloudflare Tunnel. Caddy mantiene la interfaz y la API bajo el mismo origen, utilizando el prefijo `/api` para las peticiones al backend.

Este despliegue permite enseñar el proyecto, pero depende de que el equipo anfitrión, los contenedores y la conexión estén disponibles. No equivale a alojar el entrenamiento en servidores de Cloudflare ni garantiza un servicio continuo para múltiples usuarios.

Otro aspecto del despliegue es el tamaño de los archivos enviados. Para facilitar la carga del dataset completo dentro del límite de subida del proxy, se añadió soporte para ZIP con un único CSV. En la comprobación realizada, el fichero pasó de unos 267 MB a 46 MB. El backend recupera el CSV y aplica las mismas validaciones que a una carga sin comprimir, con un máximo de 512 MiB para el contenido descomprimido. El hash se calcula sobre el CSV, de modo que la compresión no cambia la identificación del dataset ni sus datos.

El tamaño comprimido también debe respetar el límite del despliegue. Además, reducir el tamaño de la transferencia no reduce por sí solo la memoria necesaria para procesar el dataset. Si el acceso público no está disponible, la aplicación puede ejecutarse con la configuración local de Docker Compose descrita en los anexos.

Las pruebas automáticas y la integración continua acompañan estos cambios. Se comprueban casos de API, permisos, procesamiento y gestión de recursos, junto con tipos, lint y construcción del frontend. El alcance de la cobertura y las comprobaciones manuales se describe en la sección de pruebas del anexo de documentación técnica de programación. La cobertura indicada corresponde al código Python; los flujos completos y el frontend requieren sus propias comprobaciones.

6. Trabajos relacionados

Existen trabajos que estudian la clasificación de TDAH a partir de EEG con enfoques distintos. Algunos extraen características de la señal y utilizan clasificadores tradicionales, mientras que otros transforman los registros o emplean redes neuronales. En este capítulo se comentan trabajos relacionados con estas decisiones y con la idea de ofrecer una interfaz para utilizar los modelos. No se pretende realizar una revisión sistemática de toda la literatura, sino presentar varios trabajos que han abordado el tema y explicar su relación con el proyecto.

6.1. Características espectrales y clasificación

Hossain y colaboradores presentan un trabajo basado en características de distintas bandas y posiciones de los electrodos [17]. Utilizan potencia espectral y entropía, junto con un clasificador SVM, para distinguir registros de sujetos con TDAH y controles. El trabajo muestra el interés de combinar información de frecuencia y de los canales, en lugar de reducir toda la señal a un único valor.

Esta idea está relacionada con la rama ML de EEGScope, que también utiliza características espectrales, aunque las combina con medidas temporales y compara varios clasificadores. La coincidencia en el tipo de características no hace que los resultados sean directamente comparables: habría que comprobar las ventanas, las particiones y la selección de variables de ambos procedimientos. La referencia consultada es un preprint de 2025.

6.2. Potenciales relacionados con eventos

Ghasemi y colaboradores estudian la clasificación de TDAH mediante potenciales relacionados con eventos, conocidos como ERP [14]. Trabajan con registros de 60 niños, 30 con TDAH y 30 controles, obtenidos durante tareas con estímulos visuales y auditivos. Extraen características de distintas bandas de frecuencia y comparan siete algoritmos, entre ellos SVM, regresión logística y Random Forest.

El estudio analiza cómo cambia la clasificación al seleccionar y combinar características, incluidas medidas de potencia por banda. Esto conecta con la rama ML de EEGScope, donde también se construye una representación de la señal antes de entrenar los clasificadores. Sin embargo, los ERP están vinculados a estímulos concretos, mientras que el procedimiento de este proyecto utiliza ventanas multicanal sin alinearlas con eventos.

También existen diferencias en la evaluación. El artículo describe validación cruzada de diez particiones, pero no explicita una agrupación por sujeto equivalente a la utilizada en EEGScope. Por ello, sus resultados no se consideran directamente comparables con los obtenidos en este proyecto.

6.3. Redes neuronales e interfaces

Pappula y Anwar proponen una interfaz basada en espectrogramas EEG y aprendizaje profundo [27]. Su enfoque transforma la señal en una representación que se utiliza con ResNet18. Esto conecta con una parte del objetivo de este proyecto: acercar la ejecución de los modelos a una persona que no trabaja directamente con los scripts.

En EEGScope, las redes reciben ventanas multicanal y la aplicación también incorpora entrenamiento, historial de experimentos y selección de modelos. Por tanto, el interés de esta referencia no está solamente en la métrica obtenida, sino en la integración de la clasificación dentro de una herramienta. El uso de la palabra diagnóstico en el título de ese trabajo no implica que la aplicación desarrollada aquí esté validada para uso clínico.

6.4. Arquitecturas para señales EEG

EEGNet, de Lawhern y colaboradores, propone una red compacta para tareas de interfaz cerebro-computador y utiliza convoluciones específicas

para trabajar con EEG [23]. Es una referencia de arquitectura, no una demostración de clasificación de TDAH con el protocolo de este proyecto.

La implementación incluida entre los experimentos es una adaptación ligera inspirada en EEGNet. No reproduce exactamente el modelo original, por lo que sus resultados se presentan con esta aclaración. Esta diferencia evita atribuir a la arquitectura publicada las limitaciones de una implementación distinta.

6.5. Herramientas de análisis EEG

Además de los trabajos de clasificación, existen herramientas que permiten preparar, visualizar y analizar señales EEG. La Tabla 6.1 compara su orientación con la de EEGScope.

Herramienta	Entorno de uso	Funciones y orientación
EEGLAB [11]	Interfaz gráfica y scripts en MATLAB	Procesamiento y visualización de EEG, rechazo de artefactos y análisis mediante componentes independientes.
MNE-Python [26]	Biblioteca de código abierto en Python	Análisis de datos neurofisiológicos, visualización, estadística y herramientas de aprendizaje automático.
BrainVision Analyzer [2]	Aplicación gráfica comercial	Procesamiento de EEG y potenciales relacionados con eventos, con inspección visual y un historial de los pasos aplicados.
EEGScope	Aplicación web desarrollada en este TFG	Carga de CSV compatibles, entrenamiento ML y DL, registro de experimentos y predicción con modelos disponibles.

Tabla 6.1: Comparación del enfoque de distintas herramientas EEG. Elaboración propia a partir de su documentación oficial y de las funciones del proyecto.

Estas herramientas cubren necesidades de análisis más amplias que las desarrolladas en este proyecto. EEGScope se centra en ofrecer desde una interfaz web el flujo experimental de clasificación utilizado en el trabajo, con

usuarios, datasets y modelos asociados. Su alcance está limitado al formato de entrada y a los procedimientos implementados.

La comparación se refiere a las funciones documentadas. No se han realizado pruebas comparativas de rendimiento ni se considera que disponer de herramientas de análisis EEG implique una validación para diagnosticar TDAH.

6.6. Relación con el proyecto

La aportación de EEGScope no consiste en proponer un nuevo clasificador, sino en reunir una comparación experimental y una aplicación que permita completar el flujo de trabajo. Se puede cargar un dataset, entrenar, consultar resultados y utilizar un modelo disponible para inferencia. El registro de recursos, los usuarios y la ejecución en segundo plano forman parte de esa integración.

La tabla 6.2 resume los enfoques de los trabajos comentados y su relación con EEGScope. Se comparan sus entradas y objetivos, sin ordenar sus resultados, ya que los procedimientos de evaluación son distintos.

Trabajo		Enfoque	Relación con EEGScope
Hossain al. [17]	et	Características espectrales de bandas y canales EEG, clasificadas mediante SVM.	Comparte el uso de características espectrales. EEGScope las combina con medidas temporales y compara varios clasificadores.
Ghasemi al. [14]	et	Características de potenciales relacionados con eventos y comparación de clasificadores.	Permite contrastar la selección de características. EEGScope trabaja con ventanas sin alinearlas con estímulos.
Pappula y Anwar [27]		Espectrogramas EEG y ResNet18 integrados en una interfaz.	Comparte el interés por facilitar el uso de los modelos mediante una herramienta. Las redes de EEGScope reciben ventanas multicanal.
Lawhern al. [23]	et	EEGNet, una arquitectura compacta para señales EEG.	Sirve de referencia para una adaptación ligera incluida entre los experimentos. No se reproduce exactamente la arquitectura original.
EEGScope		Comparación de modelos ML y DL con separación por sujeto.	Integra carga de datos, entrenamiento, consulta de experimentos e inferencia en una aplicación web.

Tabla 6.2: Comparación de los enfoques de los trabajos relacionados y EEGScope.

Para comparar el rendimiento con otros trabajos no basta con colocar sus porcentajes en una tabla. Es necesario conocer si se separan los sujetos, si las métricas se calculan por ventana o por paciente y si la selección del modelo utiliza datos de prueba. Por este motivo, no se establece una clasificación de los trabajos citados a partir de sus métricas publicadas. La evaluación del proyecto se limita al protocolo y a los resultados descritos en el capítulo anterior.

7. Conclusiones y líneas de trabajo futuras

7.1. Conclusiones

EEGScope reúne los experimentos de clasificación de EEG y una aplicación web desde la que se pueden entrenar y utilizar los modelos. Se ha completado el flujo desde la carga de los datos hasta el entrenamiento, la consulta de resultados y la inferencia con modelos guardados. La aplicación incorpora usuarios y trabajos en segundo plano, de forma que el entrenamiento no tiene que mantenerse como una petición HTTP abierta.

En los experimentos guardados, Random Forest obtiene el mayor F1 ponderado medio de los modelos ML, con un 78,72 %, y CNN 1D el mayor de los modelos DL, con un 80,14 %. Estos valores corresponden a ventanas de señal evaluadas mediante particiones por sujeto. No son porcentajes de diagnóstico correcto de pacientes y no permiten afirmar una superioridad general de DL, ya que las dos ramas emplean preparaciones distintas.

La separación por sujeto es una de las decisiones principales del trabajo. Generar muchas ventanas aumenta el número de entradas disponibles, pero no añade pacientes independientes. Mantener esta separación permite que la prueba de cada partición se realice sobre sujetos que el modelo no ha utilizado para ajustarse, evitando la fuga de información que se produciría al incluir ventanas de un mismo sujeto tanto en entrenamiento como en prueba.

El desarrollo de la aplicación también ha puesto de manifiesto que guardar un clasificador no es suficiente para reutilizarlo. Es necesario conservar su configuración de entrada, relacionarlo con el experimento y controlar quién

puede acceder a él. La reorganización del código y la incorporación de pruebas acompañan este crecimiento y facilitan continuar el proyecto.

Cumplimiento de los objetivos

Los objetivos experimentales se han abordado mediante la preparación de los registros EEG, la extracción de características y la comparación de modelos ML y DL. Los resultados incluyen métricas de validación cruzada por sujeto, curvas ROC y análisis de importancia de características. Estas pruebas permiten describir el comportamiento de los modelos dentro del conjunto utilizado, con las limitaciones indicadas en esta memoria.

En la aplicación se ha completado el flujo de carga y análisis de datos, entrenamiento, consulta de experimentos e inferencia. El usuario puede conservar sus experimentos y utilizar los modelos que se hayan exportado correctamente. El registro y el inicio de sesión permiten asociar los recursos a sus propietarios, mientras que los trabajos en segundo plano permiten continuar utilizando la interfaz durante el entrenamiento.

Los objetivos técnicos se han trabajado mediante la separación del backend y el frontend, la persistencia en PostgreSQL y la configuración de ejecución con Docker Compose. Las pruebas automáticas y las comprobaciones de integración continua aportan evidencias sobre las partes verificadas. Su alcance se detalla en los anexos, junto con las instrucciones de instalación y uso.

7.2. Limitaciones

El estudio utiliza un conjunto de datos limitado y no incluye una validación con otro grupo independiente de sujetos. La selección de modelos se apoya en los resultados de la validación cruzada del propio conjunto, por lo que hace falta una evaluación externa antes de generalizar las conclusiones.

Las métricas por ventana pueden dar más peso a sujetos con registros largos. Además, la desviación entre particiones no es un intervalo de confianza ni una prueba de significación. Las figuras de importancia ayudan a explorar el comportamiento de los modelos, pero no justifican explicaciones clínicas o causales.

La aplicación tampoco está preparada como un servicio clínico ni como una plataforma de entrenamiento de gran escala. El despliegue depende del equipo anfitrión, el worker procesa una tarea simultánea y los resultados de

las tareas en Redis tienen una conservación temporal. La autenticación y la separación de recursos son necesarias para publicar la herramienta, pero no sustituyen a una revisión completa de seguridad y operación.

7.3. Líneas de trabajo futuras

Evaluación experimental

Una continuación sería evaluar sobre datos externos y calcular métricas por paciente con una regla de agregación definida de antemano. También sería conveniente comparar ML y DL con condiciones más próximas y utilizar una validación cruzada anidada, con particiones internas para elegir la configuración y particiones externas para evaluar el resultado.

El análisis de importancia podría repetirse con distintas particiones por sujeto, conservando la misma medida y representación para comparar los modelos. Esto permitiría estudiar si los canales destacados se mantienen o dependen demasiado de una única separación de los datos.

Gestión de trabajos y recursos

En la aplicación se podría guardar un historial permanente del estado de las tareas, permitir cancelarlas y establecer límites de uso por usuario. Estas mejoras serían útiles antes de aumentar el número de personas que entrenan a la vez. Ampliar los workers debería ir acompañado de límites de memoria y procesamiento para evitar saturar el equipo.

Otra mejora sería trasladar la aplicación, los datasets y los modelos a un servidor que no dependa del ordenador local, incorporando copias de seguridad y comprobaciones de recuperación. Así se podría acceder al sistema sin tener encendido el ordenador de desarrollo

Pruebas y mantenimiento

Queda margen para ampliar las pruebas del frontend y las pruebas de extremo a extremo de los flujos completos. También se podrían incorporar recuperación de contraseña y una gestión más completa de las sesiones. Estas tareas deben mantener la separación de datos entre usuarios y comprobar tanto los casos correctos como los errores.

En conjunto, el resultado es un prototipo académico que permite experimentar y mostrar el proceso de clasificación. Su utilidad está en hacer ese

proceso accesible y reproducible dentro de las condiciones documentadas, sin presentarlo como una herramienta de diagnóstico.

Bibliografía

- [1] Astral. uv: Working on projects. <https://docs.astral.sh/uv/guides/projects/>. Accedido: 05-09-2026.
- [2] Brain Products GmbH. BrainVision Analyzer. <https://www.brainproducts.com/solutions/analyzer/>. Información oficial del producto. Consultado: 15-09-2026.
- [3] Celery Project. Introduction to celery. <https://docs.celeryq.dev/en/stable/getting-started/introduction.html>. Accedido: 05-09-2026.
- [4] Tianqi Chen and Carlos Guestrin. XGBoost: A scalable tree boosting system. <https://arxiv.org/abs/1603.02754>, 2016. arXiv:1603.02754.
- [5] Cloudflare. Cloudflare pages: Git integration. <https://developers.cloudflare.com/pages/configuration/git-integration/>. Accedido: 05-09-2026.
- [6] Cloudflare. Cloudflare tunnel. <https://developers.cloudflare.com/cloudflare-one/networks/connectors/cloudflare-tunnel/>. Accedido: 05-09-2026.
- [7] Mike X Cohen. *Analyzing Neural Time Series Data: Theory and Practice*. The MIT Press, 2014. <https://mitpress.mit.edu/9780262019873/analyzing-neural-time-series-data/>.
- [8] danizo. EEG Dataset for ADHD. <https://www.kaggle.com/datasets/danizo/eeg-dataset-for-adhd>. Copia utilizada en el pro-

- yecto. Origen identificado: EEG data for ADHD / Control children, IEEE DataPort.
- [9] Docker. Docker compose documentation. <https://docs.docker.com/compose/>, 2026. Accedido: 23-06-2026.
 - [10] EEG data for ADHD / Control children. IEEE DataPort. <https://ieee-dataport.org/open-access/eeeg-data-adhd-control-children>.
 - [11] EEGLAB. EEGLAB Wiki. <https://eeglab.org/>. Documentación oficial. Consultado: 15-09-2026.
 - [12] Concepción Escofet Soteras, Manuel Antonio Fernández Fernández, Carme Torrents Fenoy, Fernando Martín del Valle, Gonzalo Ros Cervera, and Irene Sofía Machado Casas. Trastorno por déficit de atención e hiperactividad (TDAH). In *Protocolos diagnósticos y terapéuticos de Neurología Pediátrica*, volume 1, pages 85–92. Asociación Española de Pediatría, 2022. https://static.aeped.es/pdf_final_protocolos_aep_senep_2022_c0942dd6e4.pdf.
 - [13] FastAPI. Fastapi documentation. <https://fastapi.tiangolo.com/>, 2026. Accedido: 23-06-2026.
 - [14] Elham Ghasemi, Mansour Ebrahimi, and Esmaeil Ebrahimie. Machine learning models effectively distinguish attention-deficit/hyperactivity disorder using event-related potentials. *Cognitive Neurodynamics*, 16(6):1335–1349, 2022. <https://pmc.ncbi.nlm.nih.gov/articles/PMC9666608/>.
 - [15] GitHub. Github actions documentation. <https://docs.github.com/actions>, 2026. Accedido: 23-06-2026.
 - [16] GitHub. Github docs. <https://docs.github.com/>, 2026. Accedido: 23-06-2026.
 - [17] Md Bayazid Hossain, Md Anwarul Islam Himel, Md Abdur Rahim, Shabbir Mahmood, Abu Saleh Musa Miah, and Jungpil Shin. Classification of ADHD and healthy children using EEG based multi-band spatial features enhancement. <https://arxiv.org/abs/2504.04664>, 2025. Preprint, arXiv:2504.04664.
 - [18] Keras team. Bidirectional layer. https://keras.io/api/layers/recurrent_layers/bidirectional/. Documentación oficial.

- [19] Keras team. Conv1D layer. https://keras.io/api/layers/convolution_layers/convolution1d/. Documentación oficial.
- [20] Keras team. LSTM layer. https://keras.io/api/layers/recurrent_layers/lstm/. Documentación oficial.
- [21] Keras team. Save, serialize, and export models. https://keras.io/guides/serialization_and_saving/. Documentación oficial. Consultado: 15-09-2026.
- [22] Keras team. Training & evaluation with the built-in methods. https://keras.io/guides/training_with_built_in_methods/. Documentación oficial. Consultado: 15-09-2026.
- [23] Vernon J. Lawhern, Amelia J. Solon, Nicholas R. Waytowich, Stephen M. Gordon, Chou P. Hung, and Brent J. Lance. EEGNet: A compact convolutional network for EEG-based brain-computer interfaces. <https://arxiv.org/abs/1611.08024>, 2018. arXiv:1611.08024, versión 4.
- [24] Meta Open Source. React documentation. <https://react.dev/>, 2026. Accedido: 23-06-2026.
- [25] Mind Media. 10-20 eeg system. https://help.mindmedia.com/en_US/articles/420512-10-20-eeg-system. Documentación del fabricante. Consultado: 15-09-2026.
- [26] MNE Developers. MNE-Python documentation. <https://mne.tools/stable/index.html>. Documentación oficial. Consultado: 15-09-2026.
- [27] Medha Pappula and Syed Muhammad Anwar. An ADHD diagnostic interface based on EEG spectrograms and deep learning techniques. <https://arxiv.org/abs/2412.02695>, 2024. Preprint, arXiv:2412.02695.
- [28] PostgreSQL Global Development Group. Postgresql documentation. <https://www.postgresql.org/docs/>, 2026. Accedido: 23-06-2026.
- [29] pytest. pytest documentation. <https://docs.pytest.org/>, 2026. Accedido: 23-06-2026.
- [30] pytest-cov. pytest-cov documentation. <https://pytest-cov.readthedocs.io/>, 2026. Accedido: 23-06-2026.
- [31] scikit-learn developers. Cross-validation: evaluating estimator performance. https://scikit-learn.org/stable/modules/cross_validation.html. Documentación oficial. Consultado: 05-09-2026.

- [32] scikit-learn developers. Metrics and scoring: quantifying the quality of predictions. https://scikit-learn.org/stable/modules/model_evaluation.html. Documentación oficial. Consultado: 05-09-2026.
- [33] scikit-learn developers. Permutation feature importance. https://scikit-learn.org/stable/modules/permutation_importance.html. Documentación oficial. Consultado: 05-09-2026.
- [34] scikit-learn developers. Supervised learning. https://scikit-learn.org/stable/supervised_learning.html. Documentación oficial. Consultado: 15-09-2026.
- [35] SciPy developers. `scipy.signal.welch`. <https://docs.scipy.org/doc/scipy/reference/generated/scipy.signal.welch.html>. Documentación oficial. Consultado: 05-09-2026.
- [36] Software Freedom Conservancy. Git documentation. <https://git-scm.com/doc>, 2026. Accedido: 23-06-2026.
- [37] SonarSource. Sonarqube cloud documentation. <https://docs.sonarsource.com/sonarqube-cloud/>, 2026. Accedido: 23-06-2026.
- [38] Streamlit. Streamlit documentation. <https://docs.streamlit.io/>. Documentación oficial. Consultado: 15-09-2026.
- [39] Vite. Vite documentation. <https://vite.dev/>, 2026. Accedido: 23-06-2026.
- [40] Peter D. Welch. The use of fast Fourier transform for the estimation of power spectra: A method based on time averaging over short, modified periodograms. *IEEE Transactions on Audio and Electroacoustics*, 15(2):70–73, 1967.
- [41] ZenHub. Zenhub help center. <https://help.zenhub.com/>, 2026. Accedido: 23-06-2026.